

Project 1: Warmup to C and Unix programming

Käännöksessä käytetty komento ja sen argumentit:

```
gcc reverse.c -o reverse -std=c99 -Wall -pedantic
```

```
gcc reverse.c -o reverse -std=c99 -Wall -Wextra -pedantic -g
```

```
valgrind --leak-check=full ./reverse
```

Lyhyt kuvaus

Tein ohjelman ohjeistuksen mukaan huomioiden kaikki mainitut ehdot ja myös lisähuomiot. Toteutettu ohjelma ottaa komentoriviargumentteja `main()` funktiolle, joka saa ne muodossa `argc` (argumenttien määrä) ja `argv[]` (argumentit listassa). Ohjelman toiminnallisuus on kääntää syötetyt rivit käänteiseen järjestykseen, ja syöte voi olla joko tiedostosta tai stdin syötteestä. Stdin syötteessä ohjelma ottaa rivin kerrallaan, kun painetaan lopuksi Enter ja kun haluaa lopettaa stdin syötteen, painetaan Ctrl + D -näppäinyhdistelmää. Jos ohjelmalle annettiin kirjoitettava tiedosto, tulos menee sinne, muussa tapauksessa tulos piirtyy konsoliin näytölle.

Ohjelmaa ajetaan käyttäen komentoja (olettaen, että sijainti on oikeassa kansiossa):

```
./reverse
```

```
./reverse input.txt
```

```
./reverse input.txt output.txt
```

Ohjelma alustaa aina muuttujat ensin NULLiksi, koska C-kielen kurssilla demonstroitiin, kuinka sinne muistiin voi aina jäädä jotakin ylimääräistä ja on turvallisempi tehdä asia näin. Ohjelma myös sekä reagoi virheisiin, että sisältää ohjelmanannosta poikkeavaa käyttäjän opastusta. Koodiin on jätetty pois kommentoidut printit, jolla ohjelman toiminnallisuutta voi testata tarkemmin. Ohjelma käyttää linkitettyä listaa, vaikka sen olisi voinut toteuttaa ilmankin. Tämä sen takia, että ohjeiden vinkeissä kerrottiin, että ohjeiden tekijät toteuttivat sen niin, joten itsekin halusin tehdä sen niin.

Toiminnallisuudesta

(Koodi on myös ylikommentoitu, joten sitä pitäisi pystyä lukea melkein kuin kirjaa)

Ohjelman saadessa pätevät argumentit tai ne puuttuvat, ohjelma etenee if/else if/else -rakenteella seuraavaan vaiheeseen, reverse() funktioon. If/else if/else -rakenne tarkistaa, että annetut ehdot täyttyvät, ja jos ei täyty, antaa virheilmoituksen ja poistuu tietyllä koodilla (1). Nämä ehdot ovat mm. liian monta argumenttia, liian vähän argumentteja, virheellinen syötetiedosto tai molemmat tiedostojen nimet ovat samat.

Seuraavassa vaiheessa reverse()-funktio saa parametreina osoittimet tiedostonimien merkkijonoihin, eli voi melkein sanoa, että saa "nimet", vaikkakin oikeasti parametrit ovat "mistä muistin sijainnista nimet löytyvät". Seuraavaksi tapahtuu alustus, pyrin alustamaan kaiken aina ohjelman alussa samassa kohtaa, koska se on LUT-tyyliohjeen mukaista. Koodissa on kommentoitu mitä ne muuttujat ovat.

Seuraavaksi (rivillä 36 eteenpäin) tarkistetaan, onko luettava tiedosto annettu, ja jos on, tarkistetaan, voiko sitä avata. Jos ei voi avata, annetaan virheilmoitus. Jos luettavaa tiedostoa ei ole annettu, opastetaan käyttäjää antamaan syötteen stdin kautta.

Seuraavaksi luetaan annettu tiedosto linkitettyyn listaan, varaten muistia mallocia (memory allocation) käyttäen. Jos malloc epäonnistuu, siitä annetaan virheilmoitus, kuten ohjeistettu. Valitsin While-silmukan, koska tehtävänannossa sanottiin, ettei saa olettaa tiedoston pituutta, ja tähän while-silmukka on hyvä. Sama myös merkkijonojen pituudesta, ei saanut olettaa, joten käytin osoitinta.

Tallennus tapahtuu LIFO (Last In First Out) -tyyliin, koska ohjelmalla on hyvin rajallinen toiminnallisuus, eikä sitä ole tarkoitettu laajennettavaksi, joten kannattaa pitää asiat mahdollisimman yksinkertaisina (ja luennollakin puhuttiin tästä). LIFO on siitä hyvä, että se tallentaa juuri haluttuun järjestykseen linkitettyyn listaan. Toinen vaihtoehto olisi ollut tallentaa FIFO (First In Last Out) ja sitten lukea listaa pPrevious-osoittimella, mutta sitten Structia olisi pitänyt laajentaa, ja ohjelmoijan perussääntö on "Keep it simple, stupid" (KISS), sekä ensimmäisellä ohjelmointikurssilla ensimmäinen opetettu asia: "Ohjelmoija on aina laiska". Kolmas vaihtoehto olisi ollut kääntää listan sisältö, mutta helpoin on LIFO, joten sillä mentiin.

Kun tiedosto on tallennettu linkitettyyn listaan, getline()-funktion käyttämä muisti vapautetaan ja luettava tiedosto suljetaan. Seuraavaksi rivillä 78 otetaan talteen listan alkuun osoittava osoitin, koska myöhemmin tarvitsee vapauttaa listan käyttämän muistin ja siirrän alkuperäistä osoitinta myöhemmin, joten se ei enää tule osoittamaan listan alkuun.

Seuraavaksi ohjelma tarkistaa onko output-tiedosto annettu, ja jos on, voikoi sitä avata/luoda onnistuneesti. Epäonnistuminen antaa opastetun virheen ja poistuu exit(1) käyttäen. Jos tiedosto on annettu ja siihen onnistutaan kirjoittamaan, käydään linkitettyä listaa while-silmukalla läpi hyödyntäen osoitinta "ptr" ja fprintf() funktiota, kuten opastettiin ohjeissa. Parametreina toimii tiedoston nimi, sisällön tyyppi, ja tallennettu rivi. Rivillä 91 siirretään osoitinta seuraavaan solmuun, ja näin edetään koko lista läpi. Kun lista on kuljettu läpi tallentaen sen sisällön, kirjoitettava tiedosto suljetaan. Rivillä 97 on else-haara, jos kirjoitettavaa tiedostoa ei annettu, ja se kirjoittaa samaan tyyliin näytölle, fprintf():ssä määritellään vain ulostulo stdout:iin.

Lopuksi rivillä 105 lähtien, vapautetaan linkitetyn listan käyttämä muisti, hyödyntäen väliaikaista osoitinta, jotta osoittimen muistia vapauttaessa ei katoaisi tieto seuraavasta linkitetyn listan solmun osoitteesta. Kun muisti on vapautettu, palataan reverse()-funktioista main()-funktioon ja kiitetään ohjelman käytöstä, kuten ohjelmointikursseilla on opastettu, eli LUT ohjelmointityyli sekä poistutaan ohjelmasta.

Testaus ja toiminnallisuus

Ohjelman testaamisessa käytetyt komennot:

`gcc reverse.c -o reverse -std=c99 -Wall -Wextra -pedantic -g`

Ilman argumentteja testi: `valgrind --leak-check=full ./reverse`

```
root@LEO-PC:~/Käyt Sys/Projekti1# valgrind --leak-check=full ./reverse
==77153== Memcheck, a memory error detector
==77153== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==77153== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==77153== Command: ./reverse
==77153==
No input or output file names given, using stdin and stdout in later phase.

INSTRUCTIONS:
No input file given, write each text row one by one, pressing Enter after each.
When you want to continue, press ctrl + D.

Hello
World
Test

No write file name given, writing to the screen.
In reverse the given rows are:

Test
World
Hello

Thank you for program use.
==77153==
==77153== HEAP SUMMARY:
==77153==   in use at exit: 0 bytes in 0 blocks
==77153==   total heap usage: 9 allocs, 9 frees, 2,576 bytes allocated
==77153==
==77153== All heap blocks were freed -- no leaks are possible
==77153==
==77153== For lists of detected and suppressed errors, rerun with: -s
==77153== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Invalid file testi: `valgrind --leak-check=full ./reverse wrong.txt`

```
root@LEO-PC:~/Käyt Sys/Projekti1# valgrind --leak-check=full ./reverse wrong.txt
==77484== Memcheck, a memory error detector
==77484== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==77484== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==77484== Command: ./reverse wrong.txt
==77484==
error: cannot open file 'wrong.txt'
==77484==
==77484== HEAP SUMMARY:
==77484==   in use at exit: 0 bytes in 0 blocks
==77484==   total heap usage: 1 allocs, 1 frees, 472 bytes allocated
==77484==
==77484== All heap blocks were freed -- no leaks are possible
==77484==
==77484== For lists of detected and suppressed errors, rerun with: -s
==77484== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Yhdellä oikealla argumentilla testi: `valgrind --leak-check=full ./reverse input.txt`

```

root@LEO-PC:~/Käyt Sys/Projekti1# valgrind --leak-check=full ./reverse input.txt
==77772== Memcheck, a memory error detector
==77772== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==77772== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==77772== Command: ./reverse input.txt
==77772==

No write file name given, writing to the screen.
In reverse the given rows are:

a file
is
this
hello

Thank you for program use.
==77772==
==77772== HEAP SUMMARY:
==77772==    in use at exit: 0 bytes in 0 blocks
==77772==   total heap usage: 12 allocs, 12 frees, 6,256 bytes allocated
==77772==
==77772== All heap blocks were freed -- no leaks are possible
==77772==
==77772== For lists of detected and suppressed errors, rerun with: -s
==77772== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Kahdella oikealla argumentilla testi: valgrind --leak-check=full ./reverse input.txt output.txt

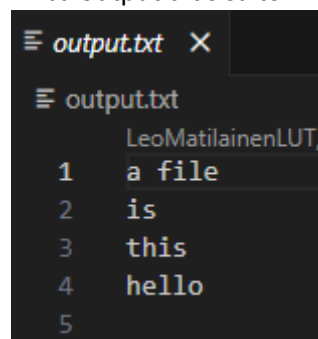
```

root@LEO-PC:~/Käyt Sys/Projekti1# valgrind --leak-check=full ./reverse input.txt output.txt
==78731== Memcheck, a memory error detector
==78731== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==78731== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==78731== Command: ./reverse input.txt output.txt
==78731==

Thank you for program use.
==78731==
==78731== HEAP SUMMARY:
==78731==    in use at exit: 0 bytes in 0 blocks
==78731==   total heap usage: 14 allocs, 14 frees, 10,824 bytes allocated
==78731==
==78731== All heap blocks were freed -- no leaks are possible
==78731==
==78731== For lists of detected and suppressed errors, rerun with: -s
==78731== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Ja output.txt sisältö:



```

LeoMatilainenLUT,
1 a file
2 is
3 this
4 hello
5

```

Testi todella pitkille riveille, koska oletuskohdassa mainittiin, ettei pituutta saa olettaa ja ohjelman täytyy toimia pitkilläkin riveillä.

[illegible]

Liian monen argumentin testaus: `valgrind --leak-check=full ./reverse too.txt many.txt arguments.txt`

```
root@LEO-PC:~/Käyt Sys/Projekt11# valgrind --leak-check=full ./reverse too.txt many.txt arguments.txt
==2194== Memcheck, a memory error detector
==2194== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==2194== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==2194== Command: ./reverse too.txt many.txt arguments.txt
==2194==
usage: reverse <input> <output>
==2194==
==2194== HEAP SUMMARY:
==2194==     in use at exit: 0 bytes in 0 blocks
==2194==   total heap usage: 0 allocs, 0 frees, 0 bytes allocated
==2194==
==2194== All heap blocks were freed -- no leaks are possible
==2194==
==2194== For lists of detected and suppressed errors, rerun with: -s
==2194== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```